AMRUTVAHINI COLLEGE OF ENGINEERING, SANGAMNER

(An Autonomous Institute)

S.Y. B.Tech. – Electronics and Computer Engineering

Data Structures and Algorithms | R25-EC-PCC-202

CCE III – COURSE PROJECT

UNIT V: TREES AND GRAPHS

COLLEGE CAMPUS NAVIGATION AND SHORTEST PATH FINDER USING GRAPHS

Submitted by	Submitted to
1. Siddharth Shinde 2. Yash Rathi 3. Akash Salve	Department of Electronics and Computer Engineering Amrutvahini College of Engineering

Academic Year: 2026–2027 | Semester: III

INDEX

Sr. No.	Contents
1	Abstract
2	Problem Statement and Objectives
3	Introduction and Project Overview
4	Data Structures Used and Justification
5	Campus Locations, Weighted Graph and Adjacency Matrix
6	System Design, Class Diagram and Flowchart
7	Algorithms and Their Explanation
8	C++ Source Code
9	Sample Input and Output
10	Testing and Results
11	Complexity Analysis
12	Advantages, Applications and Limitations
13	Future Scope and Conclusion
14	Viva Questions
15	References

Submission note: As per the course instructions, the theory/documentation may be handwritten. The commented source code and sample input/output should be printed and attached. The project must be demonstrated and explained by the group.

1. ABSTRACT

The College Campus Navigation and Shortest Path Finder is a menu-driven C++ application that helps students, faculty members and visitors identify routes between selected campus locations. The campus is represented as a weighted, undirected graph. Each building or facility is a vertex, each road is an edge, and the distance between connected locations is the edge weight. The application supports location display, connection display, linear searching, Breadth First Search (BFS), Depth First Search (DFS), and shortest-route calculation using Dijkstra's algorithm. The graph is stored in a manually implemented adjacency matrix, and a simple array-based queue is used for BFS. The program follows the restriction on STL containers by avoiding vector, queue, stack, map, set and list.

2. PROBLEM STATEMENT

Students and visitors who are unfamiliar with a college campus may find it difficult to locate departments, offices, the library, canteen, parking area and auditorium. Searching for these places manually can waste time. The project provides a basic computer-based representation of campus roads and calculates the shortest route between two selected locations using graph algorithms.

3. OBJECTIVES

• To represent campus buildings and facilities as vertices of a graph.

• To represent roads and distances as weighted edges.

• To implement BFS and DFS for graph traversal.

• To implement Dijkstra's algorithm for shortest-path calculation.

• To search campus locations using Linear Search.

• To develop an interactive C++ application using classes and constructors.

• To understand and demonstrate the practical use of graphs.

4. INTRODUCTION AND PROJECT OVERVIEW

A graph is a non-linear data structure made up of vertices and edges. Graphs are useful for representing networks in which objects are connected to one another. Examples include road maps, railway networks, computer networks and social networks. In this project, a campus location is a vertex and a road joining two locations is an edge. Since the road distance is stored, the graph is weighted. Since each road can be travelled in either direction, the graph is undirected.

The application starts by loading eight sample locations and nine road connections. The user then selects an operation from the menu. The program displays the result and returns to the menu until Exit is selected. The locations and distances in this report are illustrative sample data; replace them with measured campus distances if your group has access to an actual campus map.

5. DATA STRUCTURES USED AND JUSTIFICATION

5.1 Graph

A graph G = (V, E) contains a set of vertices V and a set of edges E. In this project, V contains the eight campus locations and E contains the road connections between them.

5.2 Weighted Undirected Graph

The graph is weighted because every road has a distance in meters. It is undirected because a road between two campus locations is considered usable in both directions. The program stores the same distance in adj[u][v] and adj[v][u].

5.3 Adjacency Matrix

An adjacency matrix is a two-dimensional array adj[MAX][MAX]. A positive value at adj[i][j] indicates a direct road and stores its distance. Zero indicates that there is no direct road between the two locations. The diagonal values are zero because a location is not connected to itself by a road in this sample.

5.4 Other arrays

Arrays store location names, visited flags, tentative distances, parent vertices and the queue used by BFS. DFS uses recursion instead of a separately allocated stack.

5.5 Justification

A graph is appropriate because a campus contains multiple locations connected by roads. The adjacency matrix is easy to understand and is suitable for this small network. It also makes checking a direct connection straightforward. The implementation is manual and does not use prohibited STL containers.

6. CAMPUS LOCATIONS AND WEIGHTED GRAPH

Vertex	Location
A	Main Gate
B	Library
C	Canteen
D	Computer Department
E	Mechanical Department
F	Administrative Office
G	Parking Area
H	Auditorium

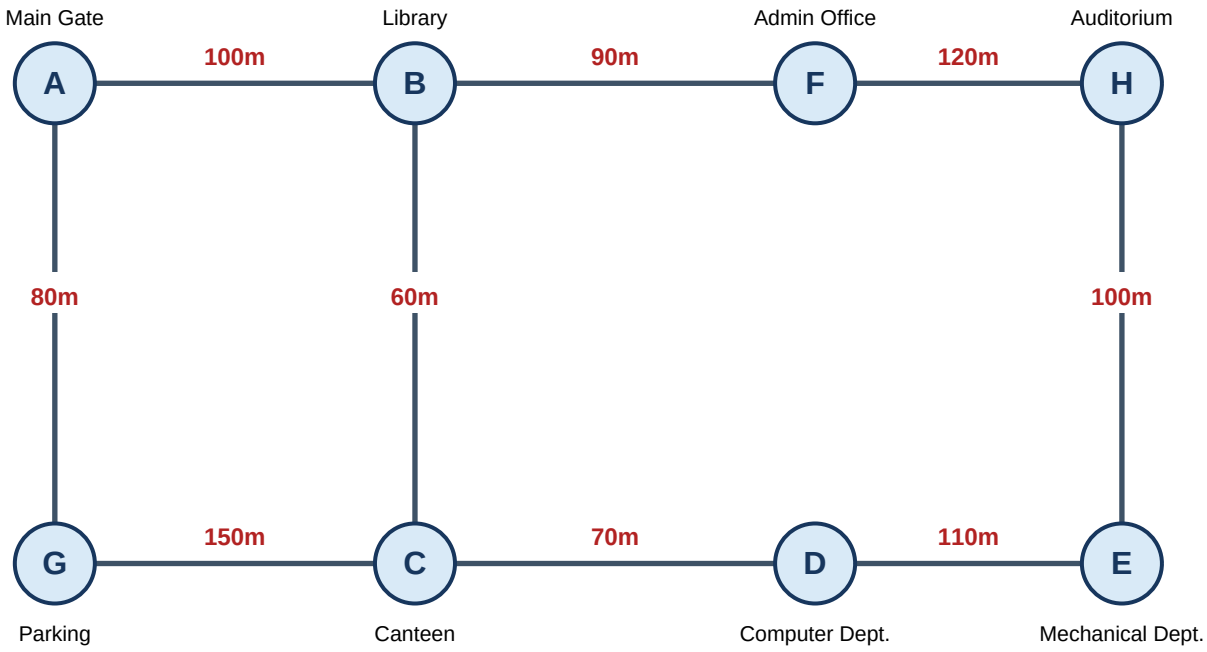


Figure 1. Weighted undirected graph of the sample campus network. Edge labels show distance in meters.

6.1 Edge List

Edge	Distance
A–B	100 m
A–G	80 m
B–C	60 m
B–F	90 m
C–D	70 m
D–E	110 m
F–H	120 m
G–C	150 m
E–H	100 m

6.2 ADJACENCY MATRIX

The matrix below follows the vertex order A, B, C, D, E, F, G, H. Each non-zero entry is the distance in meters. The matrix is symmetric because the graph is undirected.

	A	B	C	D	E	F	G	H
A	0	100	0	0	0	0	80	0
B	100	0	60	0	0	90	0	0
C	0	60	0	70	0	0	150	0
D	0	0	70	0	110	0	0	0
E	0	0	0	110	0	0	0	100
F	0	90	0	0	0	0	0	120
G	80	0	150	0	0	0	0	0
H	0	0	0	0	100	120	0	0

7. SYSTEM DESIGN

The application is divided into modules so that each operation has a clear responsibility:

Location Management: Stores and displays campus location names.

Connection Management: Adds roads and stores their distances in the adjacency matrix.

Search Module: Uses Linear Search to find a location by name.

BFS Module: Visits reachable locations level by level using an array-based queue.

DFS Module: Explores reachable locations recursively.

Shortest Path Module: Uses Dijkstra's algorithm and a parent array to print the route.

Menu Module: Accepts the user's choice and calls the required operation.

7.1 CLASS DIAGRAM

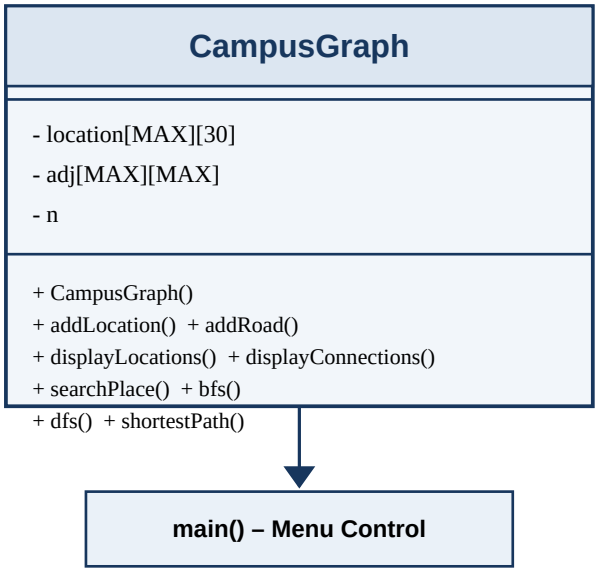


Figure 2. Class diagram showing the CampusGraph class and the main() menu controller.

7.2 FLOWCHART

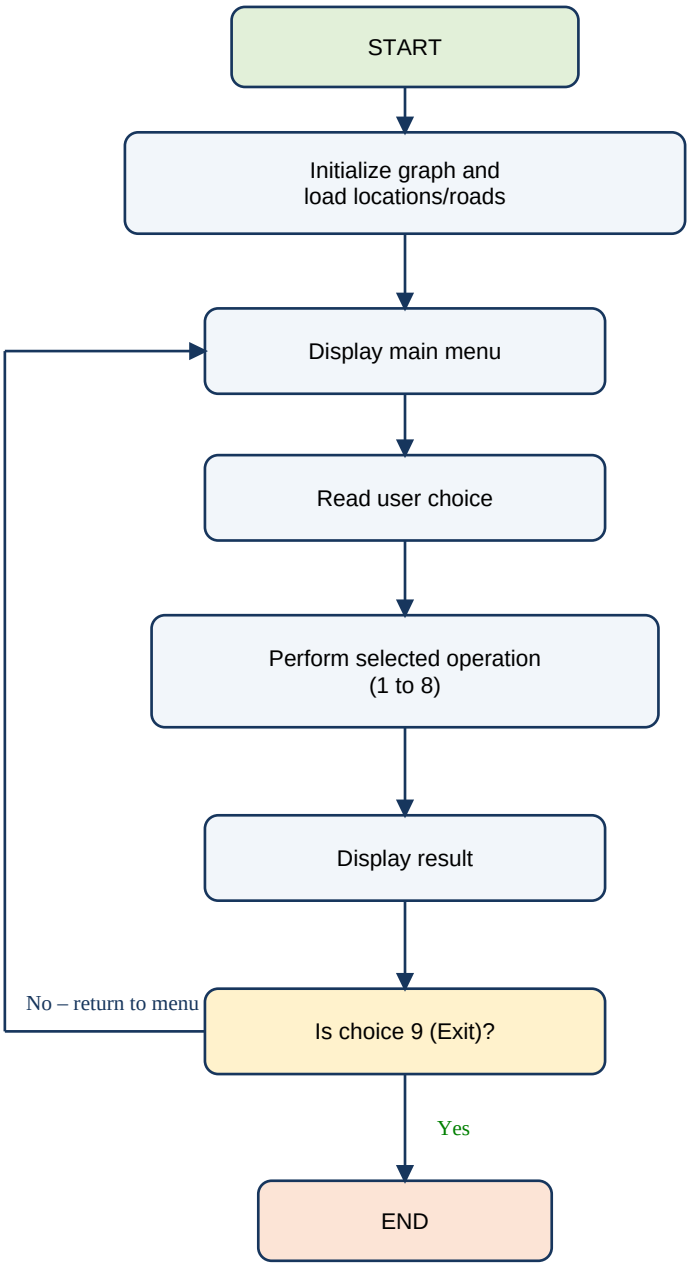


Figure 3. Program flowchart. The user returns to the menu after each operation and exits only when option 9 is selected.

8. ALGORITHMS AND EXPLANATION

8.1 Linear Search

Purpose: Find a campus location by comparing the entered name with each stored name.

• Start from index 0.

• Compare the entered name with the current location using string comparison.

• If both names match, return the index.

• If the end of the array is reached without a match, return -1.

Time complexity: O(V), where V is the number of locations.

8.2 Breadth First Search (BFS)

Purpose: Visit all locations reachable from a starting location, level by level. BFS uses a queue.

• Mark the starting vertex as visited and insert it into the queue.

• Remove the front vertex and display it.

• Check all adjacent vertices. For each connected, unvisited vertex, mark it visited and insert it into the queue.

• Repeat until the queue becomes empty.

Time complexity: $O(V^2)$ with an adjacency matrix.

8.3 Depth First Search (DFS)

Purpose: Explore a path as far as possible before backtracking. The implementation uses recursion.

• Mark the current vertex as visited and display it.

• Scan its adjacent vertices.

• For every connected, unvisited vertex, call DFS recursively.

• Return when no unvisited adjacent vertex remains.

Time complexity: $O(V^2)$ with an adjacency matrix.

8.4 Dijkstra's Shortest Path Algorithm

Purpose: Find a minimum-distance route when all edge weights are non-negative. This project uses positive road distances.

• Set all tentative distances to infinity and all parent values to -1. Set the source distance to zero.

• Select the unvisited vertex with the smallest tentative distance.

• For each adjacent unvisited vertex, check whether travelling through the selected vertex gives a shorter distance.

• If it is shorter, update the distance and record the selected vertex as the parent.

• Mark the selected vertex visited and repeat until no reachable unvisited vertex remains.

• Follow the parent array backwards from the destination to reconstruct the route.

Time complexity: $O(V^2)$ with an adjacency matrix and a linear scan for the minimum distance.

8.5 Important implementation details

The graph is undirected, so addRoad() writes the same weight in both directions. The BFS queue is implemented using a fixed-size array with front and rear indices. DFS uses recursion. Dijkstra's algorithm uses dist[], visited[] and parent[] arrays. No STL container is used.

9. COMPLETE C++ SOURCE CODE

The following program uses a manually implemented adjacency matrix and array-based BFS queue. It avoids STL containers prohibited by the course instructions. Compile with a C++11 or later compiler. Location names are entered as single words.

```
#include <iostream>
#include <climits>
#include <cstring>
using namespace std;

#define MAX 10

class CampusGraph {
private:
    char location[MAX][30];
    int adj[MAX][MAX];
    int n;

    // Linear search for a location name
    int searchLocation(char name[]) {
        for (int i = 0; i < n; i++) {
            if (strcmp(location[i], name) == 0)
                return i;
        }
        return -1;
    }

    // Recursive utility function for DFS
    void dfsUtil(int v, int visited[]) {
        visited[v] = 1;
        cout << location[v] << " ";

        for (int i = 0; i < n; i++) {
            if (adj[v][i] > 0 && !visited[i])
                dfsUtil(i, visited);
        }
    }

public:
    // Constructor initializes an empty graph
    CampusGraph() {
        n = 0;
        for (int i = 0; i < MAX; i++) {
            for (int j = 0; j < MAX; j++)
                adj[i][j] = 0;
        }
    }

    // Add a new campus location
    void addLocation(char name[]) {
        if (n >= MAX) {
            cout << "Maximum locations reached!\n";
            return;
```

```
    }
    if (searchLocation(name) != -1) {
        cout << "Location already exists!\n";
        return;
    }
    strcpy(location[n], name);
    n++;
    cout << "Location added successfully!\n";
}

// Add an undirected road with positive distance
void addRoad(char a[], char b[], int distance) {
    int u = searchLocation(a);
    int v = searchLocation(b);

    if (u == -1 || v == -1 || u == v || distance <= 0) {
        cout << "Invalid road details!\n";
        return;
    }

    adj[u][v] = distance;
    adj[v][u] = distance;
    cout << "Road added successfully!\n";
}

// Display all campus locations
void displayLocations() {
    cout << "\nCampus Locations:\n";
    for (int i = 0; i < n; i++)
        cout << i + 1 << ". " << location[i] << endl;
}

// Display every direct connection
void displayConnections() {
    cout << "\nCampus Connections:\n";
    for (int i = 0; i < n; i++) {
        cout << location[i] << " -> ";
        for (int j = 0; j < n; j++) {
            if (adj[i][j] > 0)
                cout << location[j] << "(" << adj[i][j] << "m) ";
        }
        cout << endl;
    }
}

// Search for a campus location
void searchPlace() {
    char name[30];
```

```
    cout << "Enter location name: ";
    cin >> name;

    int pos = searchLocation(name);
    if (pos == -1)
        cout << "Location not found!\n";
    else
        cout << "Location found: " << location[pos] << endl;
}

// Breadth First Search using a manual array queue
void bfs(char start[]) {
    int s = searchLocation(start);
    if (s == -1) {
        cout << "Invalid starting location!\n";
        return;
    }

    int visited[MAX] = {0};
    int q[MAX];
    int front = 0, rear = 0;

    visited[s] = 1;
    q[rear++] = s;

    cout << "BFS Traversal: ";
    while (front < rear) {
        int v = q[front++];
        cout << location[v] << " ";

        for (int i = 0; i < n; i++) {
            if (adj[v][i] > 0 && !visited[i]) {
                visited[i] = 1;
                q[rear++] = i;
            }
        }
    }
    cout << endl;
}

// Depth First Search using recursion
void dfs(char start[]) {
    int s = searchLocation(start);
    if (s == -1) {
        cout << "Invalid starting location!\n";
        return;
    }
```

```
    int visited[MAX] = {0};
    cout << "DFS Traversal: ";
    dfsUtil(s, visited);
    cout << endl;
}

// Dijkstra's shortest path algorithm
void shortestPath() {
    char source[30], destination[30];

    cout << "Enter starting location: ";
    cin >> source;
    cout << "Enter destination: ";
    cin >> destination;

    int s = searchLocation(source);
    int d = searchLocation(destination);

    if (s == -1 || d == -1) {
        cout << "Invalid location!\n";
        return;
    }

    int dist[MAX], visited[MAX] = {0}, parent[MAX];

    for (int i = 0; i < n; i++) {
        dist[i] = INT_MAX;
        parent[i] = -1;
    }
    dist[s] = 0;

    for (int count = 0; count < n; count++) {
        int u = -1;
        int minimum = INT_MAX;

        // Select the unvisited vertex with minimum distance
        for (int i = 0; i < n; i++) {
            if (!visited[i] && dist[i] < minimum) {
                minimum = dist[i];
                u = i;
            }
        }

        if (u == -1)
            break;

        visited[u] = 1;
```

```
            // Relax all adjacent edges
            for (int v = 0; v < n; v++) {
                if (adj[u][v] > 0 && !visited[v] &&
                    dist[u] != INT_MAX &&
                    dist[u] + adj[u][v] < dist[v]) {
                    dist[v] = dist[u] + adj[u][v];
                    parent[v] = u;
                }
            }
        }

        if (dist[d] == INT_MAX) {
            cout << "No path available!\n";
            return;
        }

        // Reconstruct path using the parent array
        int path[MAX], count = 0, current = d;
        while (current != -1) {
            path[count++] = current;
            current = parent[current];
        }

        cout << "\nShortest Path: ";
        for (int i = count - 1; i >= 0; i--) {
            cout << location[path[i]];
            if (i != 0)
                cout << " -> ";
        }
        cout << "\nTotal Distance: " << dist[d] << " meters\n";
    }
};

int main() {
    CampusGraph g;
    char name[30], a[30], b[30];
    int choice, distance;

    // Initial campus locations
    char places[8][30] = {
        "MainGate", "Library", "Canteen", "ComputerDept",
        "MechanicalDept", "AdminOffice", "Parking", "Auditorium"
    };

    for (int i = 0; i < 8; i++)
        g.addLocation(places[i]);

    // Initial undirected roads and sample distances
```

```
g.addRoad((char*)"MainGate", (char*)"Library", 100);
g.addRoad((char*)"MainGate", (char*)"Parking", 80);
g.addRoad((char*)"Library", (char*)"Canteen", 60);
g.addRoad((char*)"Library", (char*)"AdminOffice", 90);
g.addRoad((char*)"Canteen", (char*)"ComputerDept", 70);
g.addRoad((char*)"ComputerDept", (char*)"MechanicalDept", 110);
g.addRoad((char*)"AdminOffice", (char*)"Auditorium", 120);
g.addRoad((char*)"Parking", (char*)"Canteen", 150);
g.addRoad((char*)"MechanicalDept", (char*)"Auditorium", 100);

do {
    cout << "\n===== COLLEGE CAMPUS NAVIGATION =====\n";
    cout << "1. Display Campus Locations\n";
    cout << "2. Display Campus Connections\n";
    cout << "3. Search Campus Location\n";
    cout << "4. Find Shortest Path\n";
    cout << "5. BFS Traversal\n";
    cout << "6. DFS Traversal\n";
    cout << "7. Add New Location\n";
    cout << "8. Add New Road\n";
    cout << "9. Exit\n";
    cout << "Enter your choice: ";
    cin >> choice;

    switch (choice) {
    case 1:
        g.displayLocations();
        break;
    case 2:
        g.displayConnections();
        break;
    case 3:
        g.searchPlace();
        break;
    case 4:
        g.shortestPath();
        break;
    case 5:
        cout << "Enter starting location: ";
        cin >> name;
        g.bfs(name);
        break;
    case 6:
        cout << "Enter starting location: ";
        cin >> name;
        g.dfs(name);
        break;
    case 7:
```

```
            cout << "Enter new location: ";
            cin >> name;
            g.addLocation(name);
            break;
        case 8:
            cout << "Enter first location: ";
            cin >> a;
            cout << "Enter second location: ";
            cin >> b;
            cout << "Enter distance in meters: ";
            cin >> distance;
            g.addRoad(a, b, distance);
            break;
        case 9:
            cout << "Thank you for using the system!\n";
            break;
        default:
            cout << "Invalid choice!\n";
        }
    } while (choice != 9);

    return 0;
}
```

10. SAMPLE INPUT AND OUTPUT

Sample 1: Display campus locations

```
Enter your choice: 1

Campus Locations:
1. MainGate
2. Library
3. Canteen
4. ComputerDept
5. MechanicalDept
6. AdminOffice
7. Parking
8. Auditorium
```

Sample 2: Find the shortest path

```
Enter your choice: 4
Enter starting location: MainGate
Enter destination: ComputerDept

Shortest Path: MainGate -> Library -> Canteen -> ComputerDept
Total Distance: 230 meters
```

Sample 3: BFS traversal from MainGate

```
Enter your choice: 5
Enter starting location: MainGate

BFS Traversal: MainGate Library Parking Canteen AdminOffice
ComputerDept Auditorium MechanicalDept
```

Sample 4: Search for a location

```
Enter your choice: 3
Enter location name: Library
Location found: Library
```

Sample 5: Invalid location search

```
Enter your choice: 3
Enter location name: Hostel
Location not found!
```

Note: BFS order is determined by the order in which vertices are stored in the adjacency matrix. The output above matches the program's initial vertex order.

11. TESTING AND RESULTS

Test	Input / Operation	Expected result
1	Display locations	All eight locations displayed
2	Search Library	Location found
3	Search Hostel	Location not found
4	MainGate to ComputerDept	Shortest distance = 230 m
5	BFS from MainGate	All reachable vertices visited
6	DFS from MainGate	All reachable vertices explored
7	Add a new location	Location added if capacity allows
8	Invalid road details	Error message displayed

Result: The program is designed to display campus locations and connections, search locations, perform BFS and DFS traversals, and calculate the shortest path for the sample weighted graph. Run the code on your compiler and attach your own screenshots or printed console output after testing.

12. COMPLEXITY ANALYSIS

Operation	Time Complexity	Reason
Display locations	O(V)	Visits each location once
Linear search	O(V)	May compare all location names
Display connections	$O(V^2)$	Scans the full matrix
BFS	$O(V^2)$	Scans adjacency-matrix row for each visited vertex
DFS	$O(V^2)$	Scans adjacency-matrix row for each visited vertex
Dijkstra	$O(V^2)$	Linear minimum selection and matrix scan

Here, V represents the number of vertices (campus locations). The adjacency matrix requires $O(V^2)$ space. The fixed maximum capacity in this implementation is 10 locations.

13. ADVANTAGES

• Makes it easier to understand connections between campus locations.

• Finds a minimum-distance route for positive road weights.

• Provides location searching and graph traversal.

• Uses a simple menu-based interface.

• Demonstrates practical use of graph algorithms and object-oriented programming.

14. APPLICATIONS

• College and university campus navigation.

• Hospital building navigation.

• Shopping mall and office building navigation.

• Small road-network demonstrations.

• Learning and demonstration of graph algorithms.

15. LIMITATIONS

• The sample locations and distances are predefined.

• The application does not use GPS or a live map.

• Road closures and real-time traffic are not considered.

• The console interface accepts single-word location names.

• The program has a fixed capacity of 10 locations.

16. FUTURE SCOPE

• Add a graphical user interface and a visual campus map.

• Use GPS to identify the user's current location.

• Add live road-blockage and accessibility information.

• Support multi-word location names and larger campus networks.

• Develop a mobile application for students and visitors.

17. CONCLUSION

The College Campus Navigation and Shortest Path Finder demonstrates how graph data structures can solve a practical navigation problem. The campus is modelled as a weighted undirected graph using an adjacency matrix. BFS and DFS are used to traverse connected locations, Linear Search is used to locate a place, and Dijkstra's algorithm calculates a minimum-distance route. The project also applies C++ classes, constructors and arrays without using prohibited STL containers. Through implementation and testing, students gain practical understanding of graph representation and algorithms.

18. VIVA QUESTIONS AND ANSWERS

1. What is a graph?
A graph is a non-linear data structure consisting of vertices and edges.

2. What is a vertex?
A vertex represents an object or location in a graph.

3. What is an edge?
An edge represents a connection between two vertices.

4. What is a weighted graph?
A graph in which each edge has a value or weight, such as distance.

5. Is this graph directed or undirected?
It is undirected because each road can be travelled in both directions.

6. What is an adjacency matrix?
A two-dimensional array that stores connections and edge weights between vertices.

7. What is BFS?
Breadth First Search visits reachable vertices level by level.

8. Which data structure is used by BFS?
A queue. In this project, it is implemented manually using an array.

9. What is DFS?
Depth First Search explores one path deeply before backtracking.

10. How is DFS implemented here?
Using recursion.

11. What is Dijkstra's algorithm?
An algorithm that finds shortest paths from a source when edge weights are non-negative.

12. Why is Dijkstra suitable here?
All sample road distances are positive.

13. What is Linear Search?
A searching method that checks elements one by one.

14. What is the purpose of parent[]?
It stores the previous vertex on the best-known shortest path so the route can be reconstructed.

15. Why is STL not used?
The assignment restricts STL containers, so arrays and graph operations are implemented manually.

16. What is the time complexity of Dijkstra in this program?
$O(V^2)$, because it uses an adjacency matrix and linear minimum selection.

17. What is the space complexity of the adjacency matrix?
$O(V^2)$.

18. What is the role of the constructor?
It initializes the graph with zero locations and clears the adjacency matrix.

19. REFERENCES

1. Course handout: CCE III – Course Project, Unit V: Trees and Graphs, S.Y. B.Tech. Electronics and Computer Engineering, Amrutvahini College of Engineering, October 2026.

2. Bjarne Stroustrup, The C++ Programming Language, Addison-Wesley.

3. E. Balagurusamy, Object Oriented Programming with C++, McGraw-Hill.

4. Ellis Horowitz, Sartaj Sahni and Susan Anderson-Freed, Fundamentals of Data Structures in C.

5. C++ reference documentation: https://en.cppreference.com/

6. GeeksforGeeks, Graph Data Structure and Graph Traversal tutorials: https://www.geeksforgeeks.org/

7. Class notes and lectures for Data Structures and Algorithms, Unit V: Trees and Graphs.

Declaration for the project group: We have reviewed the data structures and algorithms used in this application. We will compile, test and explain the program during the project demonstration.

Student Signatures: _____ _____ _____